



Aula 02 - Analista de Dados

UC4: Machine Learning e Data Mining

Tema: Teoria de Machine Learning e Divisão de
Treino/Teste



Programação vs Machine Learning

- **Programação Tradicional:** Nós criamos as regras e fornecemos os dados. O computador executa e gera a resposta.

Dados + Regras → Respostas

- **Machine Learning:** Nós fornecemos os dados históricos e as respostas conhecidas. O computador estuda e gera a regra lógica.

Dados + Respostas → Regras



Como o ML encontra Padrões?

- **Relação Matemática:** O algoritmo tenta encontrar uma fórmula matemática que liga as variáveis de entrada (X) à saída (y).
- **Aproximação de Função:** O modelo busca uma função $f(X) \approx y$ de modo que a diferença entre a previsão e a realidade seja a menor possível.
- Ele descobre os melhores multiplicadores (pesos) para cada dado de entrada.

0 Loop de Ajuste Lógico

A máquina aprende através de um ciclo repetitivo:

- 1. **Palpite (Forward):** O modelo faz uma previsão inicial baseado nas configurações atuais.
- 2. **Erro (Loss):** O algoritmo mede a distância entre o palpite e a resposta real do histórico.
- 3. **Correção (Optimization):** Ele ajusta os parâmetros internos para reduzir o erro na próxima vez.

Aprendizado Supervisionado

O algoritmo é treinado em dados que já possuem as respostas corretas (rótulos).

- **Raciocínio:** Criar um mapeamento direto entre as variáveis preditoras (X) e a saída (y). O modelo é corrigido a cada erro de predição.
- **Casos de Uso e Exemplos:**
 - *Previsão de Inadimplência:* Histórico de clientes com target "Bom" ou "Mau" pagador.
 - *Filtro de Spam:* E-mails rotulados como "Spam" ou "Não-Spam".

Aprendizado Não Supervisionado

O algoritmo recebe dados brutos sem qualquer resposta correta ou rótulo predefinido.

- **Raciocínio:** Buscar padrões ocultos por similaridade. Ele agrupa dados baseando-se em proximidade e distância matemática entre as linhas.
- **Casos de Uso e Exemplos:**
 - *Segmentação de Clientes:* Agrupar clientes por comportamento de compra comum.
 - *Sistemas de Recomendação:* Agrupar usuários com gostos musicais parecidos.

Classificação vs Regressão

- **Classificação:** Previsão de valores categóricos discretos (classes/rótulos).
 - *Ex: VIP ou Regular, Fraude ou Normal, Churn ou Ativo.*
- **Regressão:** Previsão de valores numéricos contínuos (valores reais).
 - *Ex: Preço de venda, volume de vendas, faturamento futuro.*



Aprofundando em Classificação

- O objetivo é prever a qual categoria (classe) um determinado dado pertence.
- **Classificação Binária:** Quando existem apenas duas opções (Sim/Não, Fraude/Normal).
- **Classificação Multiclasse:** Quando temos 3 ou mais categorias possíveis (Ex: prever a espécie de uma flor).



Aprofundando em Regressão

- O objetivo é prever uma quantidade numérica na reta dos números reais.
- As respostas não são rótulos fixos, mas valores que flutuam de forma contínua.
- Avaliamos o modelo medindo a distância (o erro) entre a linha de previsão gerada e os pontos reais do gráfico.



O Perigo de "Decorar": Overfitting

Se o modelo decorar os dados de treino perfeitamente, ele perderá a capacidade de lidar com dados novos.

- **Overfitting (Sobreaajuste):** O modelo é excelente nos dados de treino, mas falha gravemente no mundo real.
- Queremos **generalização**, não memorização!



Divisão de Treino e Teste

Para provar que o modelo aprendeu e não apenas decorou, dividimos a base original:

- **Treino (70% - 80%):** A máquina estuda essa base para aprender os padrões.
- **Teste (20% - 30%):** Ocultada durante o treino. Funciona como a prova final com perguntas inéditas.

A Base de Dados Iris (Flores)

Usaremos um dataset clássico da estatística e de IA que já vem embutido no Scikit-Learn:

- **Registros:** 150 amostras de flores Iris.
- **Classes (y):** 3 espécies (*Setosa*, *Versicolor*, *Virginica*).
- **Características (X):** Comprimento/largura da sépala e comprimento/largura da pétala.



Cenário Prático (0 Problema)

Imagine que um botânico está no campo e encontra uma flor
Iris desconhecida.

- Ele quer identificar a espécie exata (*Setosa*, *Versicolor* ou *Virginica*).
- Fazer exames de DNA em laboratório é lento e caro.
- **A Solução:** Ele mede as sépalas e pétalas (comprimento e largura).
- O modelo de Machine Learning analisa esses dados e diz a espécie imediatamente!



O que o Dataset Iris Resolve?

- **Tipo de Aprendizado: Supervisionado** (cada flor no histórico tem o rótulo/gabarito de sua espécie real).
- **Tipo de Modelo: Classificação Multiclasse** (temos 3 classes discretas de flores a prever).
- **Estado Inicial (Conhecido):** As medidas de largura/comprimento das sépalas e pétalas.
- **O Desafio (Desconhecido):** Prever com precisão a espécie exata de uma nova flor que nunca foi medida antes.

Entendendo as Partições

Antes de escrever o código, vamos mapear as 4 matrizes geradas pelo particionamento:

- `X_train` / `y_train` : Características e respostas para o aprendizado (Treino).
- `X_test` / `y_test` : Características e respostas para a prova (Teste).

O que são as Dimensões (Shape)?

Ao executarmos o comando `.shape`, verificamos a estrutura do nosso conjunto:

- Exemplo `X_train.shape`: Retorna `(105, 4)`.
 - `105` representa a quantidade de linhas (exemplos para treino).
 - `4` representa a quantidade de colunas (features explicativas das flores).
- O target `y` possui apenas 1 dimensão (linhas), pois é uma única coluna de respostas.



Carregando a Base Iris no Python

```
from sklearn.datasets import load_iris
import pandas as pd

# Carregar o dataset embutido
iris = load_iris()

# Converter para DataFrame do Pandas
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

print("Features:", X.columns.tolist())
```



train_test_split com Scikit-Learn

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,          # 30% para teste
    random_state=42         # Semente aleatória fixa
)

print("Treino:", X_train.shape)
print("Teste :", X_test.shape)
```



Por que fixar o `random_state` ?

- A divisão de dados é feita de forma aleatória (como embaralhar cartas).
- Se não fixarmos a "semente" (`random_state`), cada execução do código gerará divisões diferentes.
- Fixar garante **reprodutibilidade**: os mesmos resultados sempre, facilitando testes e correções.



Desafio Prático da Aula 02

1. Importe o dataset Iris com `load_iris` .
2. Faça divisões de 80/20 e 70/30 variando o `random_state` .
3. Verifique as dimensões (`.shape`) de cada partição gerada.
4. Responda: O tamanho das matrizes mudou ao alterar a semente aleatória? Por quê?

Conclusão

Com os dados de treino e teste devidamente separados e isolados, estamos prontos para treinar nosso modelo de aprendizado!

Próxima Aula: Treinamento de Modelos de Classificação!